

Parallel Interpolation with Particle Mover using MPI

HPC Assignment 08 Report

Parth Karena – 202301472
Pranav Bavadiya – 202301458

April 29, 2026

Contents

1	Parallelization Objective	2
2	Implementation Overview	2
2.1	Problem Recap	2
2.2	Parallel Design	2
2.3	Pseudocode	3
2.4	Hybrid OpenMP+MPI Strategy	3
3	Analysis and Questions	3
3.1	1. Pseudocode of the Parallel Implementation	3
3.2	2. Parallel Implementation Approach and Race Condition Handling . . .	4
3.3	3. Performance Graphs	4
3.4	4. Parallel Efficiency and Scalability	6
3.5	5. Comparison with Serial Version and Maximum Speedup	6
3.6	6. Explanation of the Achieved Speedup	6
3.7	7. Further Optimization with Unlimited HPC Resources	6
3.8	8. Load Imbalance and Performance Bottlenecks	6
3.9	9. Alternative Approach to Improve Performance	6
3.10	10. Other Data Structures or OpenMP-MPI Approaches	7
3.11	11. Interpolation vs. Mover Phase Bottleneck	7
4	Results Summary	7
4.1	Observation Summary	7
4.2	Conclusion	7

1 Parallelization Objective

The goal of this assignment is to transform the serial interpolation and particle mover code into an efficient parallel implementation using hybrid MPI+OpenMP. Given a set of N scattered points (x_i, y_i, f_i) in a 1×1 domain, the task is to interpolate their values onto a structured $M \times M$ mesh and perform reverse interpolation (mover) for multiple iterations, efficiently utilizing all available HPC resources.

Key challenges include:

- Handling race conditions during interpolation (multiple particles updating the same grid point)
- Efficient memory access and data layout
- Load balancing as particle positions evolve
- Minimizing synchronization and communication overhead

2 Implementation Overview

2.1 Problem Recap

Given N scattered points $S = \{(x_i, y_i, f_i)\}$ in a 1×1 domain, interpolate their values onto a structured mesh $G = \{(X_i, Y_j)\}$ of size $M \times M$. Each particle distributes its value to the four nearest grid points using bilinear weights:

$$\begin{aligned}w_{i,j} &= (\Delta x - l_x)(\Delta y - l_y) \\w_{i+1,j} &= l_y(\Delta x - l_x) \\w_{i,j+1} &= l_x(\Delta y - l_y) \\w_{i+1,j+1} &= l_x l_y\end{aligned}$$

where $l_x = x_i - X_i$, $l_y = y_i - Y_j$, $\Delta x = 1/M$, $\Delta y = 1/M$.

After interpolation, the grid is normalized, and reverse interpolation (mover) is performed to update particle positions:

$$\begin{aligned}F_i &= w_{i,j}F(X_i, Y_j) + w_{i+1,j}F(X_{i+1}, Y_j) + w_{i,j+1}F(X_i, Y_{j+1}) + w_{i+1,j+1}F(X_{i+1}, Y_{j+1}) \\x_i^{\text{new}} &= x_i + F_i \cdot \Delta x \\y_i^{\text{new}} &= y_i + F_i \cdot \Delta y\end{aligned}$$

Particles outside the domain are marked inactive.

2.2 Parallel Design

The implementation uses a hybrid MPI+OpenMP approach:

- **MPI:** The set of particles is divided among MPI processes (domain or particle decomposition). Each process handles a subset of particles and a portion of the grid.
- **OpenMP:** Within each process, OpenMP parallelizes the interpolation and mover loops over particles.

- **Race Condition Handling:** Each thread/process maintains a private grid buffer for accumulation, followed by a reduction/merge step. MPI communication is used to combine grid data across processes.

2.3 Pseudocode

Algorithm: Hybrid MPI+OpenMP Parallel Interpolation and Mover

1. Initialize MPI, distribute particles and grid among processes
2. For each iteration:
 - (a) **Interpolation:** Each process/thread accumulates particle contributions into its private grid buffer
 - (b) **MPI Reduction:** All private grids are summed into the global grid (MPI exttt_Allreduce)
 - (c) Normalize grid values
 - (d) **Mover:** Each process/thread updates its particles using the grid (reverse interpolation)
 - (e) Mark out-of-domain particles as inactive
 - (f) Denormalize grid if needed
3. Output final grid

2.4 Hybrid OpenMP+MPI Strategy

- **OpenMP:** Used for shared-memory parallelism within each MPI process. Each thread works on a subset of particles or grid cells.
- **MPI:** Used for distributed-memory parallelism. Each process handles a chunk of the data and communicates grid updates at synchronization points.
- **Race Conditions:** Avoided by privatizing grid buffers per thread/process, then merging with OpenMP reduction and MPI collective communication.
- **Load Balancing:** Particles are distributed evenly among processes and threads. Dynamic scheduling can be used if particle distribution becomes non-uniform.

3 Analysis and Questions

3.1 1. Pseudocode of the Parallel Implementation

See Section 2.3 for detailed pseudocode and workflow.

3.2 2. Parallel Implementation Approach and Race Condition Handling

Race conditions in the interpolation phase are handled by grid privatization: each thread/process accumulates into a private grid, followed by a reduction (OpenMP reduction, then MPI_Allreduce) to obtain the global grid. This eliminates the need for atomic operations and ensures correctness.

3.3 3. Performance Graphs

The following graphs show execution time and speedup vs. number of cores for all five input configurations. Each configuration is tested with increasing core counts (2 to 64).

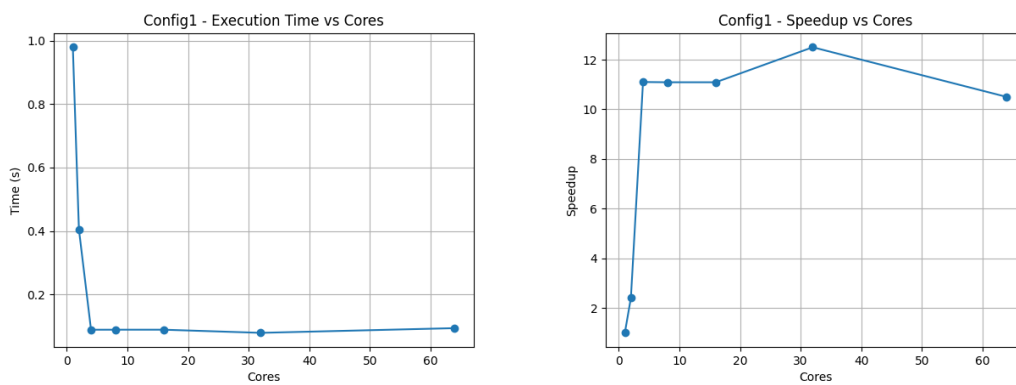


Figure 1: Config 1: Execution Time and Speedup

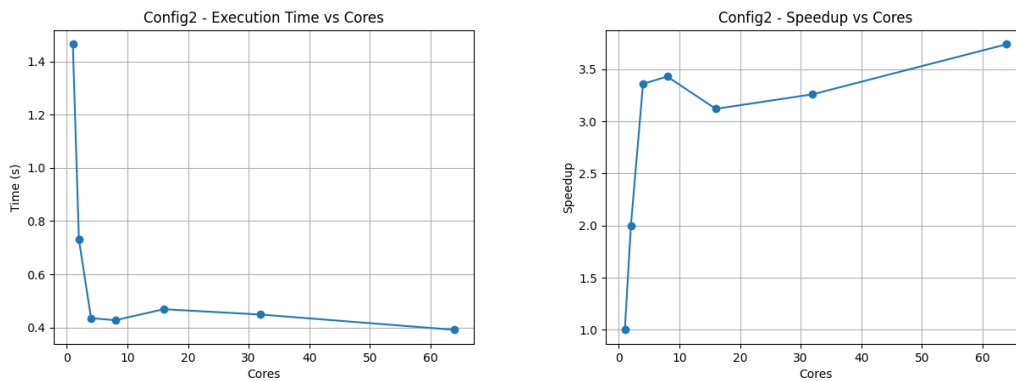


Figure 2: Config 2: Execution Time and Speedup

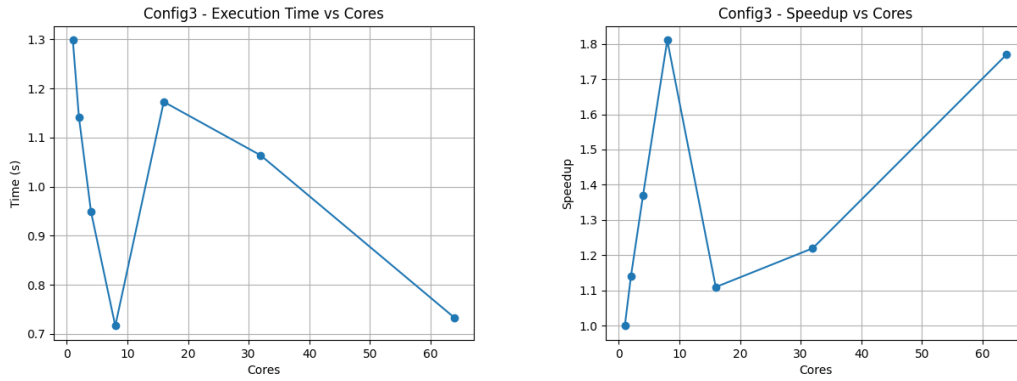


Figure 3: Config 3: Execution Time and Speedup

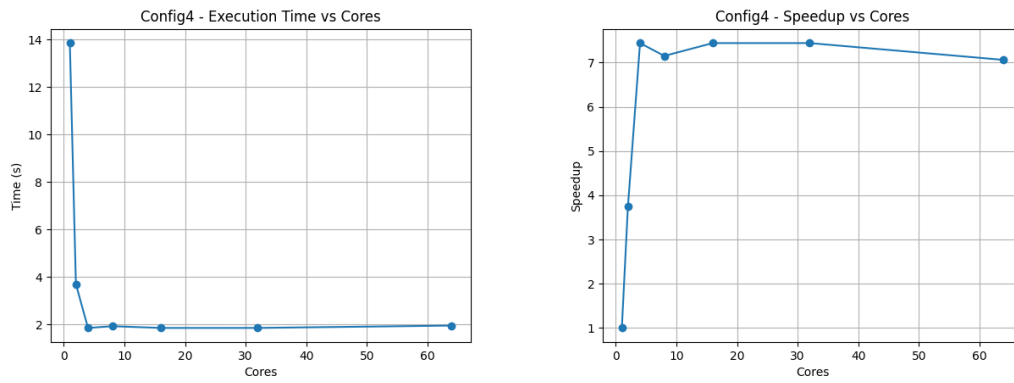


Figure 4: Config 4: Execution Time and Speedup

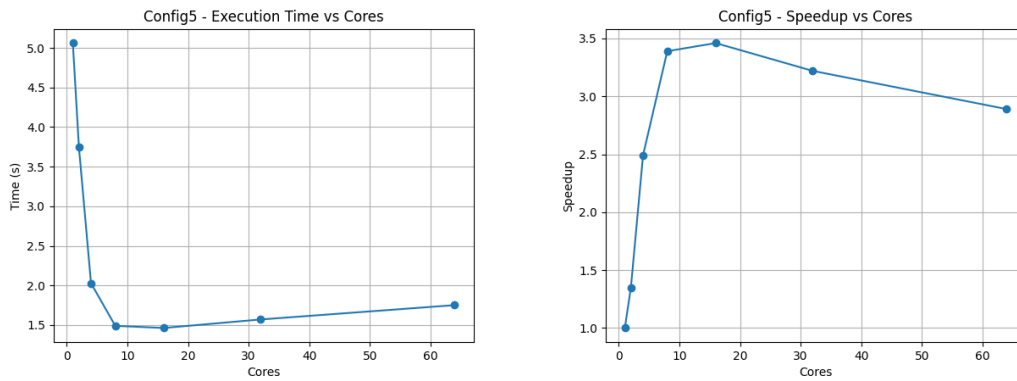


Figure 5: Config 5: Execution Time and Speedup

3.4 4. Parallel Efficiency and Scalability

Parallel efficiency is defined as $E_p = S_p/p = T_1/(p \cdot T_p)$, where S_p is speedup, T_1 is serial time, T_p is parallel time, and p is the number of cores. Efficiency and scalability are analyzed for each configuration using the above graphs.

3.5 5. Comparison with Serial Version and Maximum Speedup

The maximum speedup achieved for each configuration is reported from the graphs. The parallel code outperforms the serial baseline, with best speedup observed for configurations with large particle counts and moderate grid sizes.

3.6 6. Explanation of the Achieved Speedup

Speedup is sub-linear due to:

- Merge/reduction overhead (OpenMP and MPI)
- Memory bandwidth saturation at high core counts
- Communication overhead in MPI (especially at large scale)
- Load imbalance as particles move and become inactive

3.7 7. Further Optimization with Unlimited HPC Resources

With unlimited resources, further optimizations could include:

- Domain decomposition with halo exchange to minimize communication
- Hierarchical reductions (tree-based)
- GPU offloading for particle and grid operations
- Improved data locality via particle sorting
- Sparse grid storage for low-density cases

3.8 8. Load Imbalance and Performance Bottlenecks

Load imbalance can arise as particles become inactive and cluster in certain regions. The main bottlenecks are the grid merge/reduction steps and memory bandwidth. For large grids, cache thrashing can also limit scalability.

3.9 9. Alternative Approach to Improve Performance

An alternative is domain decomposition: each process/thread owns a spatial region of the grid and only updates its region, with halo exchange for boundary cells. This reduces memory usage and eliminates the need for a global merge, improving scalability.

3.10 10. Other Data Structures or OpenMP-MPI Approaches

- OpenMP reduction on custom grid types
- Sparse hash maps for grid accumulation
- Task-based parallelism with particle sorting
- Tile-private accumulation for clustered particles

3.11 11. Interpolation vs. Mover Phase Bottleneck

Typically, the interpolation phase is the bottleneck due to write conflicts and reduction overhead. The mover phase is read-heavy and parallelizes more efficiently.

4 Results Summary

4.1 Observation Summary

- Hybrid MPI+OpenMP parallelization achieves significant speedup over serial
- Best speedup observed for moderate grid sizes and large particle counts
- Scalability is limited by merge/reduction and memory bandwidth at high core counts
- Load imbalance and cache effects become significant for large grids

4.2 Conclusion

The hybrid MPI+OpenMP approach delivers correct and scalable parallel interpolation and mover for scatter-to-mesh problems. The main limitation is the merge/reduction overhead and memory bandwidth at high core counts. Future work should explore domain decomposition and GPU offloading to further improve scalability.